# Pomme — Modèle de données V1

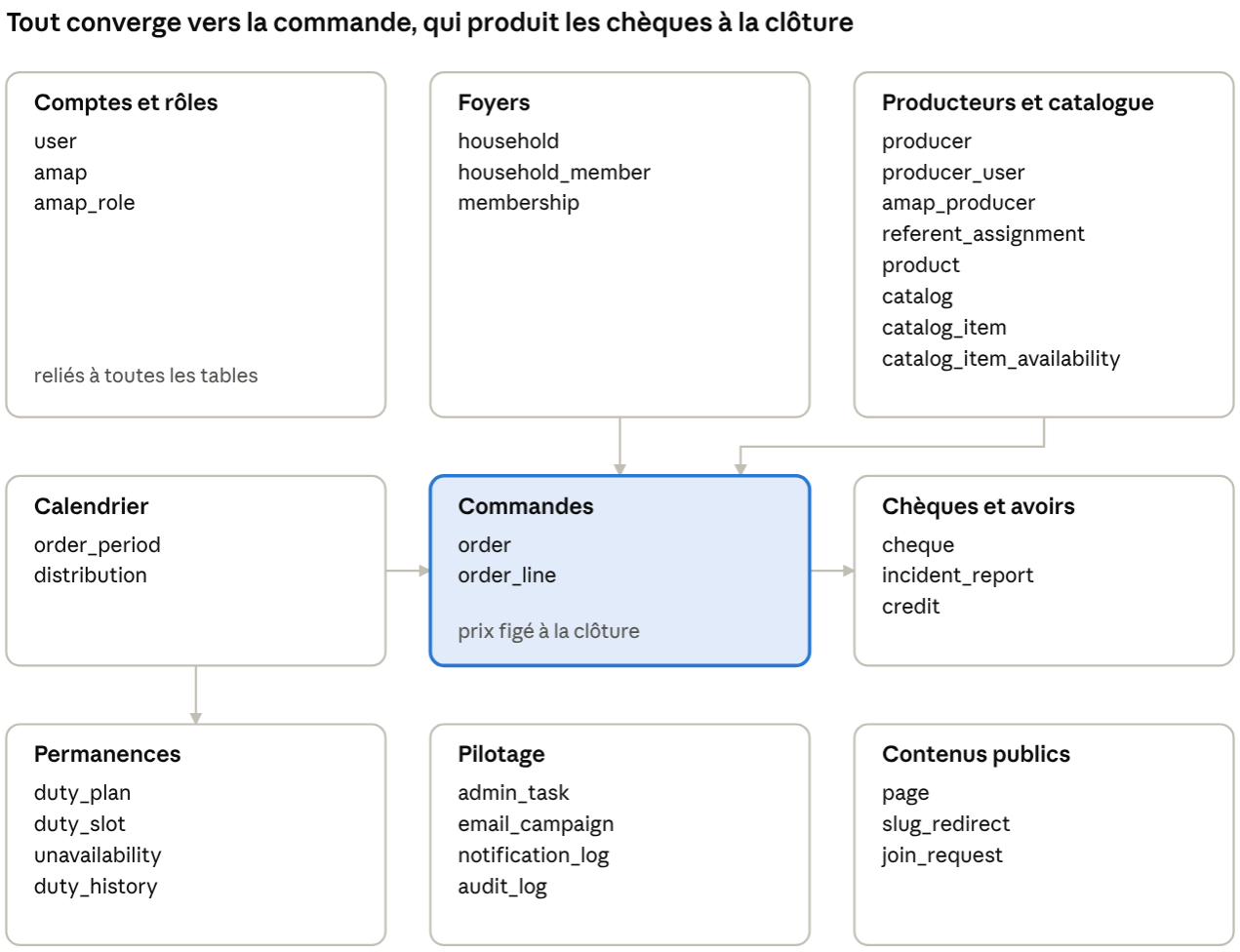
Nov 4, 2026 · @Ama

Ce modèle couvre la V1 (MVP Cambremer) pour PostgreSQL. Il est multi-AMAP dès le départ et prépare les abonnements (V2) sans les implémenter. Il découle du cahier des charges fonctionnel v0 (rév. 31).

## 1. Principes

- **Multi-AMAP** : toute table métier porte un `amap_id`, sauf les données communes à la plateforme (utilisateurs, producteurs, fiches produit). L'isolation est garantie par la base (Row Level Security PostgreSQL), en plus des contrôles applicatifs.
- **Nommage** : tables et colonnes en anglais, `snake_case`, au singulier (`household`, `order_line`). Le document donne le terme métier français à côté.
- **Identifiants** : UUID v7 (triables par date), jamais d'identifiant séquentiel exposé dans les URL.
- **Montants** : entiers en centimes (`*_cents`), jamais de nombre décimal.
- **Dates** : `date` pour les jours (distribution, adhésion), `timestamptz` pour les instants (ouverture, clôture, pointage). Fuseau Europe/Paris porté par l'AMAP.
- **Pas de suppression** des données métier : archivage (`archived_at`). Seule exception, la suppression RGPD d'un foyer, qui anonymise.
- **Horodatage** : `created_at`, `updated_at` partout ; `created_by` sur les données saisies par un humain.
- **Statuts dérivés** calculés plutôt que stockés quand ils dépendent de la date (statut d'adhésion, catalogue en lecture seule, tâches « Bloquant »).
- **Journal** : toute action financière ou correction écrit une ligne dans `audit_log`.

# 2. Schéma d'ensemble




domaines du modèle · 32 tables

Le foyer commande dans le catalogue du mois, pour des distributions ouvertes ; à la clôture, les commandes produisent un chèque attendu par foyer et par producteur, avoirs déduits. Les permanences s'appuient sur les mêmes distributions.

# 3. Comptes, AMAP et rôles

Le compte est unique sur la plateforme ; les rôles sont attribués AMAP par AMAP. Chaque ligne de `amap_role` dit « telle personne a tel rôle dans telle AMAP » : une même personne peut être administratrice à Cambremer et simple amapienne dans une autre AMAP, avec un seul identifiant de connexion. Ses droits changent selon l'AMAP où elle navigue ([domaine]/hommesdeterre ou une autre). Dans une même AMAP, un utilisateur peut cumuler les rôles : amapien, référent et administrateur.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `user` (utilisateur) | email, first_name, last_name, phone, email_verified_at, is_platform_admin | email unique ; téléphone au format international (+33…) pour les liens Signal |
| `amap` (AMAP) | slug ( `hommesdeterre` ), name, logo_url, theme (JSON : couleurs clés Material Design 3 primaire, secondaire, tertiaire, plus neutre et neutre variante facultatives ; police des titres), contact_email, created_by, distribution_place, timezone, settings (JSON : 3 bénévoles, seuil expérimenté 6, alerte adhésion 30 jours) | slug unique |
| `amap_role` (rôle) | amap_id, user_id, role ( `admin` , `referent` ) | unique (amap, user, role) |

Le rôle « amapien » n'est pas stocké ici : il découle de l'appartenance à un foyer de l'AMAP. Le rôle « producteur » découle de `producer_user` (section 5). Les sessions et jetons de connexion sont gérés par la brique d'authentification (tables `session` , `verification_token` , passkey, two_factor, hors de ce modèle). Pas de mot de passe : connexion par lien magique ou passkey (décidé le 2026-11-04).

**Création d'une AMAP (story J1, décidé le 2026-11-04)** : réservée aux utilisateurs `is_platform_admin` . Une seule transaction crée la ligne `amap` , ses pages obligatoires (accueil, mentions légales), le compte du premier administrateur s'il n'existe pas encore et sa ligne `amap_role` ( `admin` ), puis envoie l'invitation. Le slug est contrôlé : minuscules, chiffres et tirets, unique, et différent des mots réservés de l'application ( `admin` , `plateforme` , `producteur` , `api` , `connexion` …) ; seul le gestionnaire peut le modifier ensuite, l'ancienne adresse redirigeant alors. L'admin d'AMAP modifie le reste des réglages (story A4). Aucune table nouvelle : le total reste de 32.

## 4. Foyers et adhésions

Le foyer est l'unité qui commande et paie ; il appartient à une seule AMAP. Une personne membre de deux AMAP a donc deux foyers, rattachés au même compte.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `household` (foyer) | amap_id, name (« Foyer Martin »), archived_at | nom unique dans l'AMAP |
| `household_member` (membre du foyer) | household_id, user_id, is_primary | unique (household, user) ; un seul contact principal |
| `membership` (adhésion) | amap_id, household_id, start_date, end_date, fee_cents (facultatif), note | end_date > start_date ; pas de chevauchement pour un même foyer |

Statut d'adhésion calculé : **active** si aujourd'hui est entre start_date et end_date, **expire bientôt** si end_date est à moins de 30 jours, **expirée** sinon. Un foyer sans adhésion active ne peut pas commander.

## 5. Producteurs, référents et catalogue

Le producteur et ses fiches produit sont communs à la plateforme ; le prix et la disponibilité sont propres à chaque AMAP et à chaque mois. Le catalogue mensuel joue le rôle du « contrat » du cahier des charges : il porte un `mode` ( `a_la_carte` en V1, `subscription` en V2).

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `producer` (producteur) | name, category (« Produits laitiers »), short_description, description, phone, email, photo_url | commun à toutes les AMAP |
| `producer_user` (compte producteur) | producer_id, user_id | un producteur peut avoir plusieurs comptes |
| `amap_producer` (producteur dans l'AMAP) | amap_id, producer_id, check_payee (ordre du chèque), distance_km, active, archived_at | unique (amap, producer) |
| `referent_assignment` (référent) | amap_producer_id, user_id | l'utilisateur doit avoir le rôle `referent` dans cette AMAP |

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `product` (fiche produit) | producer_id, name, packaging (« boîte de 6 »), photo_url, archived_at | commun à toutes les AMAP |
| `catalog` (catalogue du mois) | amap_id, producer_id, order_period_id, mode, status ( `draft` , `published` ), published_at | unique (amap, producer, period) |
| `catalog_item` (produit au catalogue) | catalog_id, product_id, price_cents, min_quantity (facultatif, non affiché), position | unique (catalog, product) ; prix > 0 |
| `catalog_item_availability` (disponibilité) | catalog_item_id, distribution_id | une ligne = produit disponible cette semaine |

Règles portées par le modèle :

- Un catalogue passe en lecture seule quand sa période est clôturée (calculé, pas de statut stocké).
- Un produit retiré est archivé ( `product.archived_at` ) ; il reste lisible dans les catalogues passés.
- « Préparer le catalogue du mois » copie le catalogue précédent en brouillon (prix et produits), disponibilités à revoir.
- Un producteur sans ligne `referent_assignment` génère une tâche « Bloquant ».

## 6. Périodes de commande, distributions et commandes

Une période par mois et par AMAP, commune à tous les producteurs. Une commande regroupe, pour un foyer, les lignes d'un catalogue (donc d'un producteur) ; chaque ligne est une quantité pour un produit et une distribution.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `order_period` (période de commande) | amap_id, month (1er du mois), opens_at, closes_at, status ( `scheduled` , `open` , `closed` ), closed_at, reopened_at | unique (amap, month) ; opens_at < closes_at |

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `distribution` (distribution) | amap_id, order_period_id, date, start_time, end_time, is_open, closed_reason | unique (amap, date) ; motif obligatoire si fermée |
| `order` (commande) | amap_id, household_id, catalog_id, created_by, updated_by | unique (household, catalog) |
| `order_line` (ligne de commande) | order_id, catalog_item_id, distribution_id, quantity, frozen_unit_price_cents | unique (order, item, distribution) ; quantité ⩾ 0 ; produit disponible et distribution ouverte |

Règles portées par le modèle :

- Le passage `open` → `closed` (tâche planifiée à closes_at, ou action admin) fige `frozen_unit_price_cents` sur chaque ligne puis calcule les chèques (section 7), dans une seule transaction.
- « Rouvrir les commandes » repasse la période en `open` ; la clôture suivante recalcule prix figés et chèques non encore pointés. Une commande dont le chèque est déjà pointé reste verrouillée, même après réouverture (décidé le 2026-11-04). De même, les lignes d'une distribution déjà passée ne sont plus modifiables (décidé le 2026-11-04).
- Toute modification d'une commande par un référent ou un admin écrit dans `audit_log`.
- Les états de l'accueil famille se déduisent : période ouverte sans commande, ouverte avec commande, clôturée.
- L'émargement se fait en V1 sur la feuille imprimée, et rien n'est ressaisi dans l'application ensuite : aucune table ne stocke donc l'émargement. Les écarts constatés remontent par les signalements (section 7). La saisie sur téléphone (story E4, en C) viendra plus tard. La feuille PDF est générée à partir de `order_line` et `duty_slot`.

## 7. Chèques et avoirs

L'échéance et le paiement sont fusionnés dans une seule table `cheque`, puisqu'il y a toujours un seul chèque par foyer, producteur et mois. Elle est créée à la clôture avec son montant attendu, puis complétée au pointage.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `cheque` (chèque attendu et reçu) | amap_id, household_id, producer_id, order_period_id, gross_cents (total commandé), credits_cents (avoirs déduits), expected_cents, status ( `expected` , `received` , `handed_over` ), received_cents, received_at, received_by, handed_over_at, handed_over_by, cheque_number (facultatif) | unique (household, producer, period) ; expected = gross − credits, jamais négatif |
| `incident_report` (signalement) | amap_id, household_id, distribution_id, order_line_id, reason ( `missing` , `damaged` , `other` ), quantity, comment, status ( `open` , `credited` , `rejected` ), created_by, handled_by, handled_at | quantité ⩽ quantité commandée ; commentaire obligatoire si `other` |
| `credit` (avoir) | amap_id, household_id, producer_id, incident_report_id, amount_cents, status ( `created` , `deducted` ), deducted_on_cheque_id, created_by | montant > 0 ; deducted_on_cheque_id renseigné si et seulement si `deducted` |

Cycle : le signalement ( `open` ) devient un avoir `created` quand le référent le valide. À la clôture suivante, les avoirs `created` du foyer chez ce producteur sont rattachés au nouveau chèque et passent `deducted` . L'écart affiché au pointage vaut received_cents − expected_cents.

Un signalement « Autre » peut aussi donner lieu à un avoir, à l'appréciation du référent (décidé le 2026-11-04 ; le cahier des charges limitait jusqu'ici les avoirs aux produits manquants ou abîmés).

Décidé le 2026-11-04 : si le foyer ne commande rien le mois suivant chez ce producteur, l'avoir reste `created` et attend la commande suivante chez ce producteur, quel que soit le délai.

## 8. Permanences et indisponibilités

Le planning est trimestriel ; chaque distribution reçoit 3 bénévoles. Les bénévoles sont des personnes ( `user` ), pas des foyers, car leur nom et leur téléphone sont imprimés.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `duty_plan` (planning du trimestre) | amap_id, quarter_start, unavailability_deadline, status ( `collecting` , `draft` , `sent` ), sent_at | unique (amap, quarter_start) |
| `duty_slot` (permanence) | duty_plan_id, distribution_id, user_id, position (1 à 3), changed_since_notice | unique (distribution, user) ; 3 au plus par distribution |
| `unavailability` (indisponibilité) | duty_plan_id, user_id, distribution_id | unique (user, distribution) |
| `duty_history` (antériorité) | amap_id, user_id, imported_count | nombre de permanences faites avant Pomme (reprise AmapJ) |

L'expérience d'un bénévole = imported_count + permanences passées dans `duty_slot` ; « expérimenté » à partir de 6. Le bandeau « Changements enregistrés sans prévenir » s'affiche tant qu'une ligne a `changed_since_notice` à vrai ; « Prévenir maintenant » envoie les emails et remet ce champ à faux.

## 9. Signalements, tâches admin, emails et journal

Les tâches « Bloquant » ne sont pas stockées : ce sont des requêtes exécutées à l'affichage du tableau de bord (producteur sans référent, catalogue non publié à J-3 de l'ouverture, distribution sans 3 bénévoles, signalement non traité…). Elles disparaissent donc seules. Seules les tâches « À suivre » sont stockées.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `admin_task` (tâche « À suivre ») | amap_id, kind, title, link, related_type, related_id, due_date, done_at, done_by | unique (amap, kind, related_id) pour éviter les doublons |

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `email_campaign` (email groupé, H1) | amap_id, audience ( `all` , `producer_customers` ), producer_id, subject, body, sent_at, sent_by, recipient_count | — |
| `notification_log` (email envoyé) | amap_id, user_id, template (relance, rappel de clôture, planning…), sent_at, provider_id | sert à ne pas relancer deux fois |
| `audit_log` (journal) | amap_id, actor_user_id, entity_type, entity_id, action, before (JSON), after (JSON), created_at | écriture seule, jamais modifié |

Les signalements sont décrits en section 7 ( `incident_report` ).

## 10. Contenus publics

Le site public n'a qu'un type de contenu : la page, organisée en arbre (décidé le 2026-11-04). Une page sans parente est une page principale (contenu fixe, éventuellement dans le menu) ; ses sous-pages sont listées sur elle. Le blog est une page principale dont les sous-pages sont les billets : techniquement rien ne les distingue des autres pages. Tout cela entre en V1.

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `page` (page ou sous-page) | amap_id, parent_id (vide = page principale), slug, title, excerpt (résumé), cover_url (image principale), body (Markdown), status ( `draft` , `published` , `unpublished` ), published_at, author_id, is_required (accueil, mentions légales), menu_position (vide = hors menu ; pages principales seulement), children_order ( `date_desc` , `manual` ), position (ordre manuel), og_title, og_description, og_image_url | unique (amap, parent, slug) ; un seul niveau de sous-pages (validé) ; une page obligatoire ne peut pas être dépubliée |

| Table (terme métier) | Champs principaux | Contraintes |
|---|---|---|
| `slug_redirect` (ancienne adresse) | amap_id, old_path, page_id | unique (amap, old_path) |
| `join_request` (demande d'adhésion, B2) | amap_id, name, email, phone, message, household_id (foyer créé si acceptée), handled_by, status ( `new` , `contacted` , `accepted` , `declined` ) | — |

Adresse d'une page = chemin de ses slugs ( `/blog/retour-recolte-solidaire` ). Quand l'adresse d'une page publiée change (slug ou parente), l'ancienne est enregistrée dans `slug_redirect` et redirige (HTTP 301). Le dernier billet publié de l'accueil = la sous-page de Blog la plus récente.

Open Graph : chaque page produit `og:title` , `og:description` , `og:image` , `og:url` , `og:site_name` (nom de l'AMAP) et `og:type` ( `article` avec `article:published_time` pour une sous-page, `website` pour une page principale). Champs og_* vides : titre, résumé ou début du texte, image principale ou logo de l'AMAP. Les mêmes valeurs servent aux balises `title` et `meta description` .

Les cartes producteurs lisent `producer` (catégorie, description courte, photo) et `amap_producer` (distance). « Prochaine distribution » lit `distribution` .

## 11. Règles d'intégrité et calculs dérivés

Ce qui est garanti par la base (contraintes, clés étrangères, RLS) ne dépend pas du code de l'interface.

| Règle | Où elle est garantie |
|---|---|
| Un utilisateur ne lit que les données des AMAP dont il est membre | RLS sur `amap_id` (variable de session positionnée à chaque requête) |
| Un seul chèque par foyer, producteur et mois | unique (household, producer, period) sur `cheque` |
| Une quantité seulement pour un produit disponible à une distribution ouverte | clé vers `catalog_item_availability` + contrôle serveur |
| Pas de commande si adhésion expirée ou période non ouverte | contrôle serveur dans l'action « enregistrer la commande » |

| Règle | Où elle est garantie |
|---|---|
| Prix figé à la clôture | transaction de clôture ; catalogue en lecture seule ensuite |
| Un référent n'agit que sur ses producteurs ; un producteur que sur son catalogue | contrôle serveur via `referent_assignment` et `producer_user` |
| Montants cohérents | contrainte CHECK expected_cents = gross_cents − credits_cents, ⩾ 0 |
| Traçabilité des actions financières | écriture dans `audit_log` dans la même transaction |

Valeurs calculées, jamais stockées : statut d'adhésion, total par producteur pendant la saisie, « Chèques traités pour N foyers sur 40 », liste « Rien ce mois-ci » (producteurs sans commande ce mois et commandés au moins une fois sur 3 mois), écarts de pointage, tâches « Bloquant ».

## 12. Questions ouvertes

- [x] ~~Avoir sans commande le mois suivant : décidé le 2026-11-04, il attend la prochaine commande chez ce producteur (section 7).~~
- [x] ~~Réouverture après pointage : interdire la modification des commandes dont le chèque est déjà pointé : décidé le 2026-11-04 (section 6).~~
- [x] ~~Champs du chèque : la maquette n'affiche que le montant ; décidé le 2026-11-04, seul le numéro est gardé, en facultatif (émetteur et banque retirés).~~
- [x] ~~Blog, pages éditables et demande d'adhésion : décidé le 2026-11-04, en V1 (section 10).~~
- [x] ~~Catégorie et description du producteur : décidé le 2026-11-04, communes à toutes les AMAP (table producer, section 5).~~
- [x] ~~Semaines passées du mois en cours : verrouillées, décidé le 2026-11-04 (section 6, impacte `order_line`) ?~~
- [x] ~~Option « Autre » dans les signalements : décidé le 2026-11-04, ajoutée (section 7, valeur `other` de `reason`) ?~~